Mediator

Mediator Pattern

Centralize many-to-many talk

DEFINITION

Define an object that encapsulates how a set of objects interact, so they don't refer to each other directly.

Mediator turns a tangled mesh of objects talking directly into a star: every colleague reports to one mediator, and the mediator decides who reacts. Components know only the hub, never each other, so many-to-many references collapse into many-to-one.

WHY IT MATTERS

When components reference and call each other directly, every new interaction adds edges to a graph that is hard to follow and impossible to reuse a piece of. Centralizing the interaction in one mediator decouples the colleagues — each depends only on the hub — and puts the choreography in one testable place you can change without rewiring the parts. The risk: the hub accretes logic until it becomes a god object.

Vasyl Krupka · Ukraine

How it works

Mediator	The interface colleagues call — notify(sender, event).
ConcreteMediator	Coordinates the colleagues and holds their references.
Colleague	A component that knows only its mediator, not its peers.

HOW TO APPLY

1. Identify components wired directly to each other in tangled ways.
2. Define a Mediator interface with a notify(sender, event) entry point.
3. Have each colleague report events to the mediator instead of calling peers.
4. Put the routing — who reacts to what — in the ConcreteMediator alone.

> LITMUS TEST
>
> Are objects interacting in complex, tangled ways that are hard to follow or reuse? If the interaction graph itself is the problem, move the wiring into a hub.

— Smell → fix

Each widget calls the others directly — an N-to-N tangle:

```
// Checkbox calls every peer directly
okButton.enable() // knows the button
statusLabel.refresh() // and the label
priceTotal.update() // the N-to-N tangle grows
```

↓ ROUTE EVERYTHING THROUGH A HUB

```
class Dialog { // the mediator / hub
notify(sender, e) {
if (e === 'toggle') this.ok.enable()
} // all routing lives here
}
class Checkbox {
toggle() { this.hub.notify(this, 'toggle') }
}
```

Colleagues now emit events to the mediator and never reference each other; the interaction rules live in Dialog alone, where you can test and change them. Watch that the hub doesn't swell into a god object.

USE WHEN

- ✓ Complex UI forms
- ✓ Chat rooms; air-traffic-style coordination

AVOID WHEN

- ▲ Few, simple interactions

— Trade-offs & pitfalls

PROS

+ Reduces n x n coupling to n
+ Centralizes control

CONS

− The mediator can become a god object

COMMON PITFALLS

▲ God-object risk: all coordination piles into the mediator until it knows too much — keep routing thin.

▲ It trades pairwise coupling between colleagues for everyone's coupling to one hub — a single point of failure for the interaction.

▲ Don't reach for it when two or three objects interact simply — a direct reference is clearer than the indirection.

WHERE YOU SEE IT

- Chat rooms and air-traffic control (GoF's own): participants talk to the hub, never peer-to-peer.
- MediatR (.NET in-process CQRS) and Redux / NgRx stores: components dispatch to one hub that owns transitions.
- DOM event delegation: one parent listener routes events from many children.

SEE ALSO

Observer	the broadcast channel a mediator often uses; Mediator centralizes routing, Observer just notifies
Facade	also fronts a subsystem — but one-way, and it doesn't coordinate peers
Command	colleagues' messages are often reified as Commands the hub dispatches
Indirection	the GRASP principle Mediator embodies — an intermediary that decouples peers

Vasyl Krupka · Ukraine

Interview & sources

Mediator vs Observer?

Mediator centralizes who-talks-to-whom in one coordinator; Observer is decentralized one-to-many broadcast. They are often combined.

Mediator vs Facade?

Facade is a one-way simplified entry to a subsystem. Mediator manages multi-directional interaction between peers.

The main downside?

The mediator can become a god object — all interaction logic centralizes there and grows hard to maintain.

How does it reduce coupling?

It turns N-to-N references between colleagues into N-to-1: each knows only the mediator, not its peers.

REMEMBER

Colleagues stop calling each other — they call one hub that coordinates them.

SOURCES

GoF — "Design Patterns" (1994): Mediator (behavioral)

Freeman & Robson — "Head First Design Patterns" (2nd ed., 2020): Mediator (leftover patterns)